

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

Part 1:

You will be assigned one of three problems based of your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.

```
In [1]: import hashlib

def hash_to_range(input_string: str) -> int:
    hash_object = hashlib.sha256(input_string.encode())
    hash_int = int(hash_object.hexdigest(), 16)
    return (hash_int % 3) + 1
input_string = "farhad"
result = hash_to_range(input_string)
print(result)
```

1

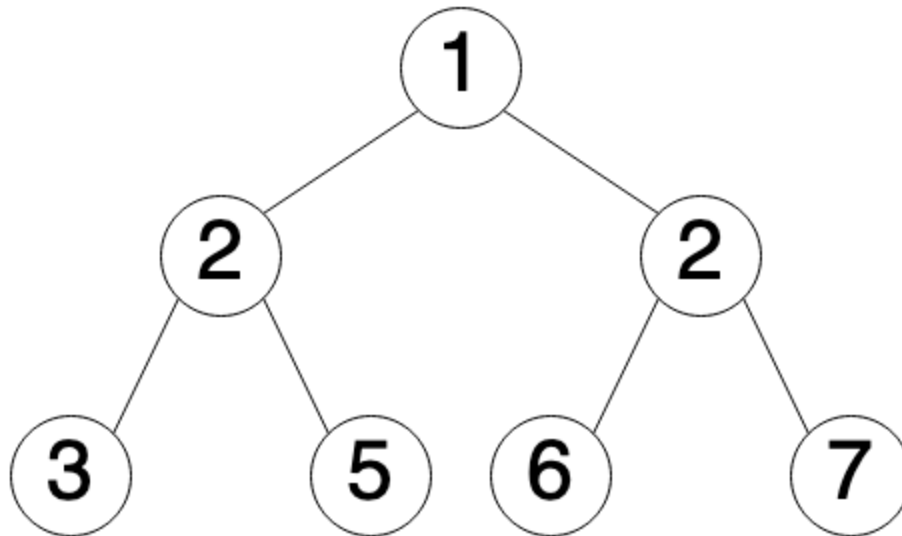
▼ Question 1

Question One: Check Duplicates in Tree

Given the `root` of a binary tree, check whether it contains a duplicate value. If a duplicate exists, return the duplicate value. If there are multiple duplicates, return the one with the closest distance to the root. If no duplicate exists, return -1.

Examples

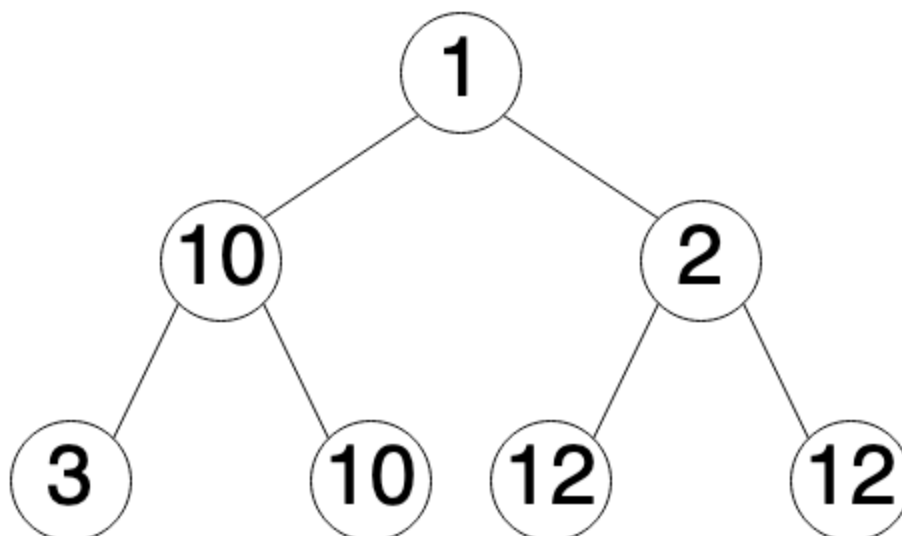
Example 1



Input: `root = [1, 2, 2, 3, 5, 6, 7]` What traversal method is this?

Output: 2

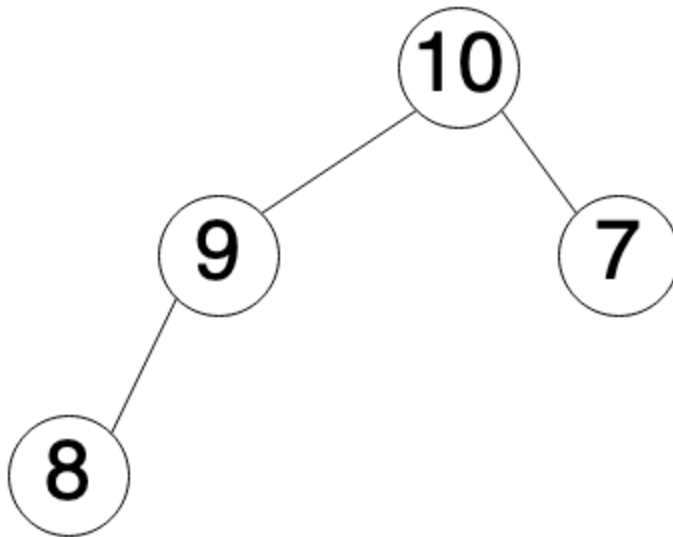
Example 2



Input: `root = [1, 10, 2, 3, 10, 12, 12]`

Output: 10

Example 3



Input: `root = [10, 9, 7, 8]`

Output: -1

Starter Code for Question 1

```

In [2]: from collections import deque
        from typing import Optional

        # Definition for a binary tree node.
        class TreeNode:
            def __init__(
                self,
                val: int = 0,
                left: Optional["TreeNode"] = None,
                right: Optional["TreeNode"] = None,
            ) -> None:
                self.val = val
                self.left: Optional["TreeNode"] = left
                self.right: Optional["TreeNode"] = right

        def is_duplicate(root: Optional[TreeNode]) -> int:
            if root is None:
                return -1
            seen: set[int] = set()
            queue = deque([root])

            while queue:
                node = queue.popleft()

                if node.val in seen:

```

```

        return node.val
    seen.add(node.val)

    if node.left is not None:
        queue.append(node.left)

    if node.right is not None:
        queue.append(node.right)

    return -1

```

Time complexity: $O(n)$, because each node is visited once.
Space complexity: $O(n)$, because the queue and seen set can store up to n nodes/vals

```

In [3]: root1 = TreeNode(val=1,
                        left=TreeNode(val=2, left=TreeNode(val=3), right=TreeNode(val=5)),
                        right=TreeNode(val=2, left=TreeNode(val=6), right=TreeNode(val=7)))
print(f"First example result is: {is_duplicate(root1)}")

root2 = TreeNode(val=1,
                left=TreeNode(val=10, left=TreeNode(val=3), right=TreeNode(val=10)),
                right=TreeNode(val=2, left=TreeNode(val=12), right=TreeNode(val=12)))
print(f"Second example result is: {is_duplicate(root2)}")

root3 = TreeNode(val=10,
                left=TreeNode(val=9, left=TreeNode(val=8)),
                right=TreeNode(val=7))
print(f"Third example result is: {is_duplicate(root3)}")

```

First example result is: 2

Second example result is: 10

Third example result is: -1

► Question 2

Starter Code for Question 2

```

In [4]: # Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def bt_path(root: TreeNode) -> list[list[int]]:
    pass

```

► Question 3

Starter Code for Question 3

```

In [5]: def missing_num(nums: list) -> int:
    pass

```

Part 2:

You and your partner must share each other's Assignment 1 submission.

```
In [6]: # Here's my partner's assignment-1 submitted link
# https://github.com/tse-e/algorithms_and_data_structures/blob/assignment-1/02_acti

# Here's my partner's assignment-1 code
# Dictionary to define the accepted bracket pairs
matches = {"(": ")", "[": "]", "{": "}"

def is_valid_brackets(s):
    opening_brackets = [] # empty stack to store unmatched opening brackets
    for i in s:            # loop for each character in the string
        if i in "([{":
            opening_brackets.append(i) # open bracket characters are pushed onto t
        elif opening_brackets == []:  # if a close bracket character is encounter
            return False
        elif matches[i] == opening_brackets[-1]: # if a close bracket characters ma
            opening_brackets.pop()
        else:
            return False
    return opening_brackets == [] # if the stack is empty at the end, True, False o
```

Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

```
In [7]: # Your answer here
# - The problem asks us to check whether a string of brackets is valid.
# A string is valid if every opening bracket is closed by the same type of bracket
# Therefore the sequences of opening and closing comply with each other for every b
# it must be closed after any other bracket types opened inside it are closed.
```

- Create 1 new example that demonstrates you understand the problem.
Trace/walkthrough 1 example that your partner made and explain it.

```
In [8]: # Your answer here
# My new example:

# Input: s = "((( [{ } ] ] ] ) ) )"
# Output: True

# My partner did two examples as below:

# Input: s = "((( ) ) )"
# Output: False
```

```
# Input: s = "[{([[]{}())}]"
```

Output: True

I would choose the shorter one, i.e. assessing why "(([]))" is not valid.

"(([]))" is not valid because all opening brackets are not closed in the correct

Walkthrough steps:

Start with an empty stack: []

Read (, push it onto the stack [(]

Read (, push it onto the stack [(, (]

Read (, push it onto the stack [(, (, (]

Read) , it matches (, so pop (from the stack: [(, (]

Read] . The top of the stack is (, but] needs [. Since they do not match, the f

Therefore string is not valid.

- Copy the solution your partner wrote.

```
In [9]: # Your answer here
# Dictionary to define the accepted bracket pairs
matches = {"(": ")", "[": "]", "{": "}"

def is_valid_brackets(s):
    opening_brackets = [] # empty stack to store unmatched opening brackets
    for i in s: # loop for each character in the string
        if i in "([{":
            opening_brackets.append(i) # open bracket characters are pushed onto t
        elif opening_brackets == []: # if a close bracket character is encounter
            return False
        elif matches[i] == opening_brackets[-1]: # if a close bracket characters ma
            opening_brackets.pop()
        else:
            return False
    return opening_brackets == [] # if the stack is empty at the end, True, False o
```

- Explain why their solution works in your own words.

```
In [10]: # Your answer here
# Because it uses a stack to track opening brackets that have not been matched yet.
# When it sees a closing bracket, it checks whether it matches the most recent open
# reverse order that they were opened. If a closing bracket does not match the top
# correctly and the stack is empty at the end, the function returns True. Else, ret
```

- Explain the problem's time and space complexity in your own words.

```
In [11]: # Your answer here
# - Time complexity: O(n) where n is the length of the input string. The function L
# Therefore, the overall time complexity is linear with respect to the input size.
```

```
# - Space complexity:  $O(n)$  in the worst case, where  $n$  is the length of the input string
# so all the opening brackets remain in the stack.
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

```
In [12]: # Your answer here
# - The solution is clear and uses the correct data structure for this problem. A stack
# last-in first-out order. The comments also make the code easier to follow.
# - One possible improvement is input handling. The code assumes that every character
# matches[i] could raise an error. This is fine if the problem guarantees that the
# check whether the character is in the matches dictionary before using it. Another
# def is_valid_brackets(s: str) -> bool:
```

Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

```
In [13]: # In Assignment 1, I focused on understanding the basic structure of each problem
# My process was slower because I was still building confidence with algorithmic thinking
# I learned that solving the problem is only one part of the assignment. I also need to
# explain my solution.

# In Assignment 2, I became more comfortable breaking problems into smaller steps.
# For Part 1, I used a breadth-first search approach because the question required
# This helped me connect tree traversal concepts to an actual coding problem instead
# I also improved my ability to compare solutions, explain time and space complexity.

# The presentation and review experience with my partner was useful because it showed
# Reviewing my partner's solution helped me notice details such as return values,
# Explaining my own solution also made me more aware of gaps in my explanation.
# Overall, Assignment 1 helped me build the foundation, while Assignment 2 helped me
```

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity

- Quality of critique of your partner's assignment, if necessary

Submission Information

🔔 Please review our [Assignment Submission Guide](#) 🔔 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
`https://github.com/<your_github_username>/algorithms_and_data_structures/`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☒ Created a branch with the correct naming convention.
- ☒ Ensured that the repository is public.
- ☒ Reviewed the PR description guidelines and adhered to them.
- ☒ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-6-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.



In []: